

# Symbolic Verification of the WireGuard Handshake and Its Post-Quantum Migration Cost

Qiyue Zhao

**Abstract.** The WireGuard handshake instantiates the Noise IKpsk2 pattern and protects production traffic in the Linux kernel. We analyse it with two symbolic provers, ProVerif and Tamarin, applied to a single transcription of the protocol, so that differences between their verdicts reflect the tools rather than the modelling. The analysis yields three main results. First, two models that differ only in whether the pre-shared key is secret isolate its contribution: against an adversary that obtains a discrete-logarithm oracle after the session ends, transport-key secrecy holds when the pre-shared key is secret and fails when it is public. Second, we locate key-compromise impersonation (KCI) resistance precisely. When the responder’s static key is compromised, the responder commits transport keys to a session that it attributes to an honest initiator, but it never completes that session; the static-ephemeral Diffie-Hellman value separates the two events. Third, we carry post-quantum parameter sizes into the network layer: an in-band ML-KEM-768 handshake initiation occupies 1300 bytes and exceeds the UDP payload guaranteed by the IPv6 minimum MTU by 68 bytes. Across 18 properties, the provers return 11 proofs, 3 attacks, 1 inconclusive result and 3 non-terminating runs. ProVerif decides 9 of its 12 properties within 24s and exhausts a 300s budget on the remaining three, whereas Tamarin decides all six of its lemmas, including both KCI levels, within 3s.

**Keywords:** WireGuard · Noise protocol framework · Symbolic verification · ProVerif · Tamarin · Post-quantum migration.

## 1 Introduction

WireGuard [5] is a virtual private network (VPN) protocol whose handshake instantiates the IKpsk2 pattern of the Noise protocol framework [13]. It is part of the mainline Linux kernel and carries production traffic, so its security properties and the cost of migrating it to post-quantum primitives have direct practical consequences.

The handshake has been analysed symbolically by Donenfeld and Milner [6] and in the computational model by Lipp, Blanchet and Bhargavan [9]. The present work is an independent reconstruction from the specification and does not report a new vulnerability. It complements prior analyses in two respects. First, the handshake is transcribed once and analysed with two symbolic provers, ProVerif and Tamarin, so that a disagreement between them is evidence about the provers rather than about two separate modelling efforts. Second, the analysis extends beyond the cryptographic layer: post-quantum parameter sizes are

carried through to datagram sizes and then to path maximum transmission units (MTUs), where the cost of migration becomes visible.

### 1.1 Contributions

This paper makes the following contributions.

- **The contribution of the pre-shared key.** The pre-shared key (PSK) of IKpsk2 is widely described as a source of post-quantum protection. We test this claim with two models that are identical except for the secrecy of the PSK, against an adversary that obtains a discrete-logarithm oracle after the session completes. Transport-key secrecy is proved in one model and refuted in the other, so the protection is attributable to the PSK (Sect. 4.2).
- **The location of KCI resistance.** When the responder’s static private key is known to the adversary, the responder commits transport keys to a session that it attributes to an honest initiator, but it never completes that session. Forging the handshake initiation requires only Diffie–Hellman values that the adversary can compute, whereas completing the session also requires the static–ephemeral value  $se = \text{DH}(e_r, S_i)$ . Key-compromise impersonation (KCI) resistance therefore resides between key commitment and session completion (Sect. 4.3).
- **The cost of post-quantum migration in three dimensions.** Carrying KEM parameter sizes through to datagram sizes and path MTUs shows that an in-band ML-KEM-768 initiation of 1300 bytes exceeds the 1232-byte UDP payload that every IPv6 path can carry without fragmentation. Together with the binding requirement on a KEM-derived PSK, this shows that the cost of migration appears simultaneously in message size, path reachability and assumption strength (Sect. 5).
- **A comparison of two provers on one transcription.** On the same model, ProVerif and Tamarin exhibit markedly different resource profiles: three ProVerif queries do not terminate within the budget, and Tamarin decides the two KCI queries among them within seconds. From this comparison we derive guidance on when each prover is the more suitable choice (Sect. 6).

### 1.2 Related Work

**Analyses of WireGuard.** Donenfeld and Milner [6] analyse the handshake and its state machine in Tamarin. Lipp, Blanchet and Bhargavan [9] give a mechanised computational proof in CryptoVerif, which yields concrete security bounds that a symbolic analysis cannot provide. Rather than strengthening a result on the same property, the present work pairs two symbolic provers on one transcription and extends the analysis to path reachability.

**Post-quantum WireGuard.** Hülsing et al. [8] redesign the handshake around key encapsulation mechanisms (KEMs). Rosenpass [15] instead runs a post-quantum key exchange over a separate channel and injects the resulting secret

into the PSK slot of WireGuard. Section 5 does not propose a further design; it provides the size arithmetic that explains why substituting ML-KEM directly into the handshake is problematic.

**KEM binding.** Cremers, Dax and Medinger [2] systematise binding properties of KEMs, and Schmieg [14] shows that ML-KEM does not achieve the strongest variants of two of them. We use this terminology in Sect. 4.4 to state which binding property a KEM-derived PSK requires.

*Organisation.* Section 2 introduces the handshake, symbolic verification and the security properties considered. Section 3 describes the models and the adversary. Section 4 reports the verification results, and Sect. 5 analyses the cost of post-quantum migration. Section 6 compares the two provers, Sect. 7 discusses limitations, and Sect. 8 concludes the paper.

## 2 Preliminaries

### 2.1 Notation

For a party  $X \in \{i, r\}$ , where  $i$  denotes the initiator and  $r$  the responder, we write  $(s_X, S_X)$  for its static key pair and  $(e_X, E_X)$  for its ephemeral key pair, with private keys in lower case. The function  $\text{DH}(x, Y)$  computes the Diffie–Hellman value of a private key  $x$  and a public key  $Y$ . We write  $\text{HASH}$  for a hash function,  $\text{KDF}_n$  for a key derivation function with  $n$  outputs,  $\text{AEAD}(k, c, m, a)$  for the authenticated encryption of a plaintext  $m$  under a key  $k$  and a counter  $c$  with associated data  $a$ , and  $\parallel$  for concatenation. Finally,  $psk$  denotes the pre-shared key and  $\varepsilon$  the empty string.

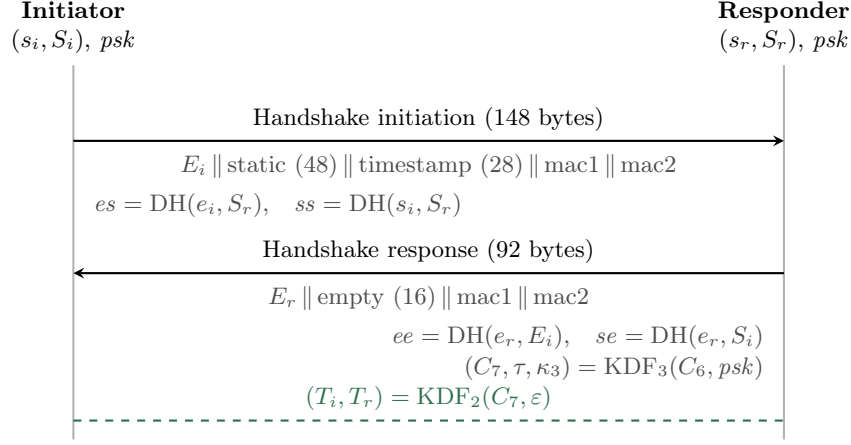
### 2.2 The WireGuard Handshake

WireGuard instantiates Noise IKpsk2 with X25519, ChaCha20-Poly1305 and BLAKE2s. The initiator knows the static public key of the responder in advance, which distinguishes the IK pattern from patterns such as XX, and the handshake completes in two messages (Fig. 1): an initiation of 148 bytes and a response of 92 bytes.

The two parties derive a common chaining key from four Diffie–Hellman values:

$$es = \text{DH}(e_i, S_r), \quad ss = \text{DH}(s_i, S_r), \quad ee = \text{DH}(e_r, E_i), \quad se = \text{DH}(e_r, S_i). \quad (1)$$

The value  $es$  binds the initiation to its intended recipient,  $ss$  provides authentication of the first message,  $ee$  contributes forward secrecy, and  $se$  authenticates the initiator to the responder in the second message. Starting from an initial



**Fig. 1.** Message flow of the WireGuard handshake.

chaining key  $C_0$ , the chaining key evolves as

$$\begin{aligned}
 C_1 &= \text{KDF}_1(C_0, E_i), & (C_2, \kappa_1) &= \text{KDF}_2(C_1, es), \\
 (C_3, \kappa_2) &= \text{KDF}_2(C_2, ss), & C_4 &= \text{KDF}_1(C_3, E_r), \\
 C_5 &= \text{KDF}_1(C_4, ee), & C_6 &= \text{KDF}_1(C_5, se), \\
 (C_7, \tau, \kappa_3) &= \text{KDF}_3(C_6, psk), & (T_i, T_r) &= \text{KDF}_2(C_7, \varepsilon).
 \end{aligned}$$

The last row mixes in the PSK and yields the transport keys  $T_i$  and  $T_r$ . The initiation encrypts the initiator's static public key under  $\kappa_1$  and a TAI64N timestamp under  $\kappa_2$ ; the response contains an authenticated encryption of the empty string under  $\kappa_3$ . A transcript hash  $H$  absorbs the responder's static public key and every ephemeral and encrypted field, and serves as associated data. Since  $\tau$  is absorbed into  $H$  before  $\kappa_3$  is used, a mismatched PSK causes authenticated decryption of the response to fail. If no PSK is configured, WireGuard uses the all-zero value.

Beyond the Noise pattern, WireGuard adds session indices, the timestamp and two MAC fields. The field `mac1` is a MAC keyed by  $\text{HASH}(L_1 \parallel S)$ , where  $L_1$  is a constant label and  $S$  is the static public key of the recipient. It demonstrates knowledge of that key and allows unsolicited messages to be discarded cheaply, but it is not a cryptographic authenticator. The field `mac2` carries a cookie when the recipient is under load.

### 2.3 Symbolic Verification

Both provers work in the Dolev–Yao model [4], in which the adversary controls the network and cryptographic primitives are perfect.

**ProVerif.** ProVerif [1] translates a protocol specified in the applied pi calculus into Horn clauses and saturates them. It is fully automatic and supports an unbounded number of sessions. Because it over-approximates the protocol, a proof is sound, whereas a reported attack may occasionally be spurious. If saturation does not converge, ProVerif produces no partial result.

**Tamarin.** Tamarin [11] specifies protocols as multiset rewriting rules and properties as formulas over execution traces. It supports mutable state, handles Diffie–Hellman exponentiation natively and analyses properties by constraint solving, which proceeds backwards from the negation of a property rather than forwards from the protocol rules.

## 2.4 Security Properties

We state all properties as trace properties over four protocol events. The initiator emits  $\text{Begin}_i(S_i, S_r)$  when it starts a session with  $S_r$ , and  $\text{Done}_i(S_i, S_r, T_i, T_r)$  when it completes the handshake. The responder emits  $\text{Commit}_r(S_i, S_r, T_i, T_r)$  after deriving the transport keys, immediately before it sends the response, and  $\text{Complete}_r(S_i, S_r, T_i, T_r)$  when it accepts the first transport record.

**Definition 1 (Transport-key secrecy).** *Transport-key secrecy holds if, for every session in which  $\text{Done}_i(S_i, S_r, T_i, T_r)$  occurs and neither static private key nor the PSK is revealed, the adversary derives neither  $T_i$  nor  $T_r$ .*

**Definition 2 (Forward secrecy).** *Forward secrecy holds if transport-key secrecy holds for every session that completes before the static private keys of both parties are revealed, while the PSK remains secret.*

**Definition 3 (Post-quantum forward secrecy).** *Post-quantum forward secrecy holds if transport-key secrecy holds for every session that completes before the adversary obtains a discrete-logarithm oracle, which returns the private key of any Diffie–Hellman public key.*

**Definition 4 (Agreement [10]).** *Let  $E_r \in \{\text{Commit}_r, \text{Complete}_r\}$ . The responder obtains non-injective agreement at  $E_r$  if every occurrence of  $E_r(S_i, S_r, \cdot, \cdot)$  is preceded by an occurrence of  $\text{Begin}_i(S_i, S_r)$ . It obtains injective agreement at  $E_r$  if, in addition, distinct occurrences of  $E_r$  are preceded by distinct occurrences of  $\text{Begin}_i$ .*

Agreement for the initiator is defined analogously, with  $\text{Done}_i$  in place of  $E_r$  and  $\text{Commit}_r$  in place of  $\text{Begin}_i$ ; the two events must then agree on both static public keys and both transport keys.

**Definition 5 (KCI resistance).** *Suppose that the adversary knows the static private key of the responder and may reveal the PSK, but does not know the static private key of the initiator. The handshake resists KCI at the commitment level if non-injective agreement at  $\text{Commit}_r$  holds, and at the completion level if non-injective agreement at  $\text{Complete}_r$  holds.*

**Definition 6 (K-PK binding [2]).** *A KEM is  $X$ -BIND-K-PK secure if no adversary can output two pairs of encapsulation key and ciphertext, with different encapsulation keys, that decapsulate to the same shared secret. The parameter  $X \in \{\text{HON}, \text{LEAK}, \text{MAL}\}$  fixes the adversary’s control over key pairs: in  $\text{HON}$ , key pairs are generated honestly and their private keys remain hidden; in  $\text{LEAK}$ , the adversary also obtains these private keys; in  $\text{MAL}$ , the adversary may construct key pairs arbitrarily.*

### 3 Modelling the Handshake

#### 3.1 One Transcription, Two Provers

The handshake is specified once as a ProVerif library, and every ProVerif query file includes this library instead of restating the protocol. The Tamarin theory is transcribed term by term from the same specification. Apart from two unavoidable divergences (Sect. 6.1), any difference between the verdicts of the two provers can therefore be attributed to the provers themselves. The shared ProVerif libraries comprise 349 lines, the eleven ProVerif query files 302 lines and the Tamarin theory 236 lines. Both models emit the events of Sect. 2.4 at the same protocol steps.

#### 3.2 Primitives

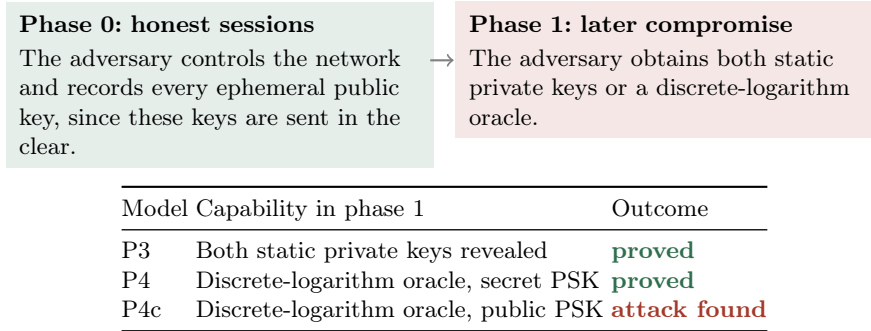
**Diffie–Hellman with two function symbols.** We model Diffie–Hellman with two function symbols rather than with iterated exponentiation. The term  $pk(x)$  denotes the public key of a private key  $x$ , and  $dh(P, y)$  denotes the shared value of a public key  $P$  and a private key  $y$ , subject to the single equation

$$dh(pk(x), y) = dh(pk(y), x). \quad (2)$$

Since  $pk$  is the only constructor of group elements, every group element that the adversary can supply has the form  $g^t$  for a term  $t$  that it can construct, which is faithful for a prime-order group. Since  $dh$  returns key material rather than a group element, a Diffie–Hellman value cannot re-enter a computation as a group element, and the depth of adversary terms is bounded by construction.

This encoding determined whether the model could be verified at all. The more natural formulation  $exp(G, x)$ , which returns a group element, allows the adversary to build towers such as  $exp(exp(exp(g, a), b), c)$  that a single commutation equation does not normalise. With that formulation, saturation of the secrecy query exhausted the memory budget after approximately  $1.2 \times 10^4$  clauses and did not terminate, whereas the two-symbol encoding proves the same query in 19 s (Table 2).

**Transcript hash.** Noise updates the transcript hash as  $H \leftarrow \text{HASH}(H \| f)$  after each field  $f$ . Since  $H$  is never transmitted and serves only as associated data, it



**Fig. 2.** Staging of adversary capabilities for models P3, P4 and P4c.

suffices that the hash after each step binds the transcript prefix injectively. We therefore model the hash after each step as a distinct collision-resistant function of the entire prefix. This flat formulation binds the same data as the iterated one but keeps term depth constant. Authenticated encryption, the key derivation chain and the MAC are modelled as ideal primitives, with one function symbol for each output of a multi-output key derivation.

### 3.3 Adversary Model

The adversary controls the network. Compromise is never built into the model: each class of secret is revealed only by an explicit action, so every query states the compromises that it tolerates.

**Compromise after the fact.** Forward secrecy and post-quantum forward secrecy concern compromise that occurs after sessions complete. We express such compromise with the phase mechanism of ProVerif (Fig. 2). In phase 0, honest sessions run against a Dolev–Yao adversary, which can record every ephemeral public key because these keys are sent in the clear. Phase 1 begins after all phase-0 sessions. In this phase the adversary receives both static private keys (P3) or a discrete-logarithm oracle (P4 and P4c). The oracle is a private function in phase 0 and becomes available only through a phase-1 process, so an adversary that recorded phase-0 traffic can invert every recorded public key. This is the standard formalisation of an adversary that records traffic now and decrypts it once a quantum computer becomes available.

**Compromise within the scenario.** KCI requires the opposite arrangement: the adversary holds the static private key of the responder while sessions run. This is an ordering constraint on individual events rather than a global staging, so we state the KCI properties as Tamarin lemmas over action facts. In these lemmas, the adversary may also reveal the PSK, and only the static private key of the initiator is protected.

**Table 1.** Wire format of the handshake messages and the corresponding model terms.

| Offset                                  | Length | Field                              | Model term                            |
|-----------------------------------------|--------|------------------------------------|---------------------------------------|
| <i>Handshake initiation (148 bytes)</i> |        |                                    |                                       |
| 0                                       | 1      | <code>message_type</code>          | —                                     |
| 1                                       | 3      | <code>reserved</code>              | —                                     |
| 4                                       | 4      | <code>sender_index</code>          | —                                     |
| 8                                       | 32     | <code>unencrypted_ephemeral</code> | $E_i = pk(e_i)$                       |
| 40                                      | 48     | <code>encrypted_static</code>      | $AEAD(\kappa_1, 0, S_i, H_1)$         |
| 88                                      | 28     | <code>encrypted_timestamp</code>   | $AEAD(\kappa_2, 0, ts, H_2)$          |
| 116                                     | 16     | <code>mac1</code>                  | $MAC(HASH(L_1    S_r), \cdot)$        |
| 132                                     | 16     | <code>mac2</code>                  | —                                     |
| <i>Handshake response (92 bytes)</i>    |        |                                    |                                       |
| 0                                       | 1      | <code>message_type</code>          | —                                     |
| 1                                       | 3      | <code>reserved</code>              | —                                     |
| 4                                       | 4      | <code>sender_index</code>          | —                                     |
| 8                                       | 4      | <code>receiver_index</code>        | —                                     |
| 12                                      | 32     | <code>unencrypted_ephemeral</code> | $E_r = pk(e_r)$                       |
| 44                                      | 16     | <code>encrypted_nothing</code>     | $AEAD(\kappa_3, 0, \varepsilon, H_5)$ |
| 60                                      | 16     | <code>mac1</code>                  | $MAC(HASH(L_1    S_i), \cdot)$        |
| 76                                      | 16     | <code>mac2</code>                  | —                                     |

### 3.4 Correspondence with the Wire Format

The value of a formal model depends on how faithfully it corresponds to the protocol that it describes. Table 1 maps each field of the two handshake messages to the corresponding term in the shared ProVerif library, where  $H_j$  denotes the transcript hash after its  $j$ -th update. Across the four WireGuard message types (initiation, response, cookie reply and transport data), 7 of the 25 fields have a model term, and each of the other 18 is omitted for a stated reason.

**Fields outside the model.** The message type and the reserved bytes serve demultiplexing; the models use separate channels instead. Session indices are chosen freely by the sender and carry no security obligation, although they enable endpoint roaming. The field `mac2` and the cookie mechanism serve load shedding, and a Dolev–Yao adversary has no notion of computational cost, so a symbolic model cannot express the property that they provide. The data-plane counter and its anti-replay window lie outside the handshake.

**An argued property.** Because `mac1` is keyed by  $HASH(L_1 || S_r)$ , an adversary that does not know the static public key of the responder cannot produce an initiation that passes the first check, and therefore cannot force the responder to perform a Diffie–Hellman computation. This property is genuine but lies outside the symbolic model; we argue it here rather than claim that the models establish it.

**Table 2.** Verification outcomes of the 18 properties.

| ID              | Property                                    | Time               | Outcome         |
|-----------------|---------------------------------------------|--------------------|-----------------|
| <i>ProVerif</i> |                                             |                    |                 |
| P1              | Transport-key secrecy                       | 19 s               | proved          |
| P3              | Forward secrecy                             | 20 s               | proved          |
| P4              | Post-quantum forward secrecy, secret PSK    | 20 s               | proved          |
| P4c             | Post-quantum forward secrecy, public PSK    | 20 s               | attack found    |
| P5a             | Initiator agreement                         | 300 s <sup>†</sup> | non-terminating |
| P5b             | Responder agreement at completion           | 23 s               | proved          |
| P5c             | Injective responder agreement at completion | 24 s               | proved          |
| P6a             | KCI resistance at commitment                | 300 s <sup>†</sup> | non-terminating |
| P6b             | KCI resistance at completion                | 300 s <sup>†</sup> | non-terminating |
| P12a            | KEM-derived PSK secrecy, binding KEM        | <1 s               | proved          |
| P12b            | KEM-derived PSK secrecy, non-binding KEM    | <1 s               | attack found    |
| P12c            | Soundness of the responder’s PSK view       | <1 s               | inconclusive    |
| <i>Tamarin</i>  |                                             |                    |                 |
| S0              | Executability of an honest run              | 3 s                | proved          |
| S1              | Transport-key secrecy                       | 2 s                | proved          |
| S3              | Forward secrecy                             | 2 s                | proved          |
| S5              | Responder agreement at commitment           | 2 s                | proved          |
| S6a             | KCI resistance at commitment                | 3 s                | attack found    |
| S6b             | KCI resistance at completion                | 2 s                | proved          |

<sup>†</sup> Budget exhausted without a verdict.

**Conditional identity hiding.** As Table 1 shows, `mac1` in the response is keyed by the static public key of the initiator. An observer who holds a candidate public key can therefore test whether a response is addressed to its owner. Identity hiding in WireGuard is thus conditional: it protects the identity of the initiator against an observer who does not already hold candidate keys, but not against one who does. This observation follows directly from representing `mac1` explicitly in the model.

## 4 Verification Results

We used ProVerif 2.05 and Tamarin 1.12.0. Each ProVerif query file and each Tamarin lemma was verified in a separate invocation on a single core of an Apple M1 machine with 8 GB of memory, under a wall-clock budget of 300 s. A run that exhausts the budget without a verdict is classified as non-terminating, and queries in the same file share the cost of that invocation. Table 2 lists the outcomes for all 18 properties: 11 are proved, 3 are refuted, 1 is inconclusive and 3 did not terminate.

#### 4.1 Secrecy and Authentication

Transport-key secrecy holds in both provers for an unbounded number of sessions (P1, S1). Forward secrecy also holds (P3, S3): revealing both static private keys after a session completes does not expose its transport keys. Since the PSK remains secret in both models, as Definition 2 requires, these results do not attribute forward secrecy to *ee* alone; Sect. 4.2 isolates the contribution of the PSK.

The responder obtains non-injective agreement at `Completer` (P5b), and Tamarin establishes non-injective agreement already at `Commitr` (S5). Injective agreement at `Completer` also holds (P5c) and expresses resistance to replay. The responder completes a session only after it decrypts a transport record under keys that depend on its own fresh ephemeral key, so a replayed initiation cannot produce a second completed session. The model does not represent the check that timestamps increase monotonically. P5c therefore states replay resistance at the completion level, whereas WireGuard also uses the timestamp to reject a replayed initiation before processing it.

Agreement for the initiator (P5a) did not terminate. We attempted three formulations: agreement on both static public keys and both transport keys, a weaker aliveness statement, and a variant with the `redundantHypElim` option of ProVerif enabled. Each formulation exhausted the budget. This direction requires reasoning through the authenticated decryption of the response under a key derived from the PSK, which is the deepest point of the key schedule.

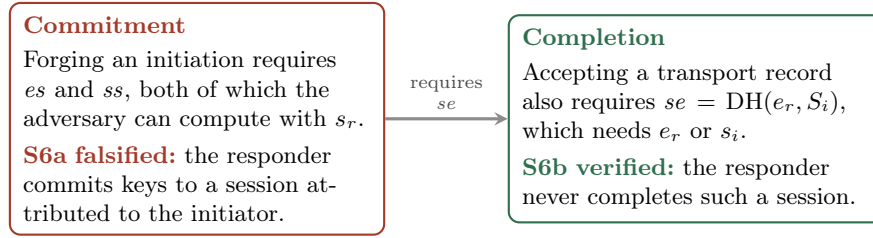
#### 4.2 The Contribution of the Pre-Shared Key

The claim that the PSK of IKpsk2 provides post-quantum protection is common but rarely tested. Models P4 and P4c are identical except that P4c publishes the PSK, which corresponds to the all-zero default of WireGuard when no PSK is configured. In both models the adversary obtains, in phase 1, a discrete-logarithm oracle that inverts every public key recorded in phase 0, including both static public keys and all ephemeral public keys.

When the PSK is public (P4c), every input to the key schedule is recoverable, and ProVerif returns a derivation that reconstructs both transport keys. When the PSK is secret (P4), the same adversary fails:  $KDF_3$  mixes a secret that is independent of all Diffie–Hellman values into the chaining key, and no discrete-logarithm computation recovers it. Since the two models differ in this single parameter, the change in outcome is attributable to the PSK. This is the precise sense in which the PSK provides post-quantum protection, and the control model turns the claim into a tested result.

#### 4.3 The Location of KCI Resistance

KCI asks whether an adversary that compromises the static private key of a party can impersonate other parties to that party. We pose this question at both levels of Definition 5, and the two levels separate (Fig. 3).



**Fig. 3.** The two levels of KCI resistance when the static key of the responder is compromised.

**The responder commits.** A well-formed initiation in the name of an honest initiator requires  $es = \text{DH}(e_i, S_r)$  and  $ss = \text{DH}(s_i, S_r)$ . The adversary chooses  $e_i$  itself and computes  $ss = \text{DH}(s_r, S_i)$  with the compromised key, so it can make the responder commit transport keys to a session that the responder attributes to the initiator. Tamarin refutes the commitment-level lemma (S6a) and returns the corresponding trace.

**The responder does not complete.** Deriving the transport keys additionally requires  $se = \text{DH}(e_r, S_i)$ , which can be computed only with the ephemeral private key of the responder or the static private key of the initiator. The adversary has neither, so it cannot produce an acceptable transport record. Tamarin verifies the completion-level lemma (S6b), although the adversary may also reveal the PSK.

**Operational consequence.** KCI resistance in WireGuard therefore relies on treating a session as established only after authenticated traffic has been received, not after the response has been sent. A model with a single session-establishment event would report one verdict where there are two, and would place the property at the wrong point. ProVerif did not terminate at either level (P6a, P6b), even with the initiator restricted to a single session.

#### 4.4 KEM-Derived Pre-Shared Keys

If the PSK is derived from a KEM instead of being configured manually, as in Rosenpass [15], the question is which property the KEM must provide. We model the essential step: the initiator encapsulates to the KEM public key of the responder, and both parties use the resulting shared secret as the PSK. We compare two symbolic KEMs. In the first, the shared secret depends on the encapsulation key and on the encapsulation randomness. In the second, it depends on the randomness alone, and a re-encapsulation operation turns a ciphertext for one encapsulation key into a ciphertext for another key with the same shared secret.

**Table 3.** Handshake message sizes in bytes with X25519 and with in-band ML-KEM substitution.

| KEM               | $ ek $ | $ ct $ | Initiation             | Response               |
|-------------------|--------|--------|------------------------|------------------------|
| X25519 (baseline) | 32     | 32     | 148                    | 92                     |
| ML-KEM-512        | 800    | 768    | 916 ( $\times 6.2$ )   | 828 ( $\times 9.0$ )   |
| ML-KEM-768        | 1184   | 1088   | 1300 ( $\times 8.8$ )  | 1148 ( $\times 12.5$ ) |
| ML-KEM-1024       | 1568   | 1568   | 1684 ( $\times 11.4$ ) | 1628 ( $\times 17.7$ ) |

**Binding determines the outcome.** With the first KEM, secrecy of the established PSK holds (P12a). With the second, the adversary re-encapsulates an observed ciphertext to an encapsulation key of its own, decapsulates the result and recovers the PSK (P12b). The attack breaks no primitive; it exploits only the missing binding between the shared secret and the encapsulation key. Since the symbolic KEM admits only well-formed key pairs, including those that the adversary generates itself, the property that separates the two models corresponds to LEAK-BIND-K-PK in the sense of Definition 6. ML-KEM achieves this notion [2], but not the stronger MAL-BIND-K-PK [14].

**An undecided query.** A third query asks whether the view of the responder is sound, that is, whether any PSK that the responder accepts was either produced by the honest initiator for the same encapsulation key or is already known to the adversary. A bare encapsulation authenticates no one, so the property must be stated as this disjunction, and the over-approximation of ProVerif could not decide it (P12c). We report the query as inconclusive rather than weakening it until it can be proved.

## 5 The Cost of Post-Quantum Migration

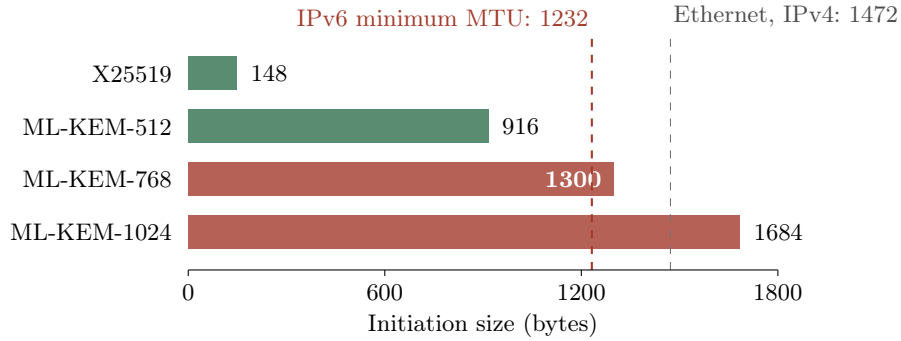
The analysis in this section involves neither verification nor measurement. Every quantity follows arithmetically from the wire-format layout of Sect. 3.4 and the parameter sizes of ML-KEM [12], so the conclusions depend neither on the machine nor on the resource limits of the provers.

### 5.1 Message Size

We consider the smallest in-band substitution: the 32-byte ephemeral public key of the initiator is replaced by an ML-KEM encapsulation key, the ephemeral public key of the responder is replaced by the corresponding ciphertext, and every other field keeps its length. The resulting sizes (Table 3) are therefore lower bounds for any in-band design that carries an ML-KEM encapsulation key in the initiation and a ciphertext in the response. The initiation grows by a factor of 8.8 with ML-KEM-768 and by a factor of 11.4 with ML-KEM-1024.

**Table 4.** Usable UDP payload of five paths and the margin left by an in-band ML-KEM-768 initiation, in bytes.

| Path                  | MTU  | Usable UDP payload | Margin |
|-----------------------|------|--------------------|--------|
| IPv6 minimum MTU      | 1280 | 1232               | −68    |
| Ethernet, IPv4        | 1500 | 1472               | +172   |
| Ethernet, IPv6        | 1500 | 1452               | +152   |
| PPPoE, IPv4           | 1492 | 1464               | +164   |
| Tunnel stacking, IPv4 | 1400 | 1372               | +72    |

**Fig. 4.** Handshake initiation size with X25519 and with in-band ML-KEM, compared with the usable UDP payload of two paths.

## 5.2 Path Reachability

Every IPv6 link must support an MTU of at least 1280 bytes [3]. After the 40-byte IPv6 header and the 8-byte UDP header, 1232 bytes remain for the UDP payload. Among the paths in Table 4, only this minimum is guaranteed; the others are common configurations. An in-band ML-KEM-768 initiation of 1300 bytes exceeds the guaranteed payload by 68 bytes (Fig. 4), so it cannot be carried without fragmentation on every IPv6 path. Packetization layer path MTU discovery for datagram transports [7] can find a larger MTU where one exists, but it cannot create one where none exists.

The consequences form a chain. KEM parameter sizes determine datagram sizes, datagram sizes must fit path MTUs, datagrams that exceed the minimum MTU depend on fragmentation or path MTU discovery, and on paths where neither succeeds the handshake cannot complete. A design that preserves a single-packet handshake must therefore transmit smaller post-quantum values than an in-band ML-KEM-768 substitution does, or move the post-quantum key exchange to a separate channel, as Rosenpass does [15].

This requirement conflicts with a deliberate property of WireGuard. The single-packet, stateless handshake needs no fragment reassembly, holds no state before authentication and makes the cookie mechanism possible; in-band migration puts exactly this property at risk. ML-KEM-512 fits every path in Table 4

with an initiation of 916 bytes, but it targets NIST security category 1, compared with category 3 for ML-KEM-768 [12]. ML-KEM-1024, with an initiation of 1684 bytes, fits none of the five paths.

### 5.3 Three Dimensions of Cost

Combining this analysis with Sect. 4.4 shows that the cost of migration has three dimensions:

- *Message size*: an in-band initiation grows by a factor of 8.8 with ML-KEM-768 and 11.4 with ML-KEM-1024.
- *Path reachability*: beyond ML-KEM-512, in-band substitution loses the guarantee of a usable path.
- *Assumption strength*: a KEM-derived PSK preserves the guarantees of a configured PSK only if the KEM binds its shared secret to the encapsulation key.

These costs arise simultaneously, so a design that addresses only one of them does not address the cost of migration as a whole.

## 6 Comparison of the Provers

Since both provers analyse the same transcription, the differences reported in this section reflect the tools. Table 5 aggregates the outcomes of Table 2 by prover.

### 6.1 Divergent Transcriptions

Two differences between the transcriptions were unavoidable, and both arise from the treatment of Diffie–Hellman.

**Term construction.** ProVerif requires the two-symbol encoding of Sect. 3.2 to bound term depth, whereas Tamarin normalises Diffie–Hellman terms natively and uses its built-in theory. The two formulations agree on the equational theory but differ in the terms that the adversary can construct.

**Expression of compromise.** ProVerif expresses compromise with phases, and Tamarin with action facts. Phases impose a global order, which suits the post-quantum experiment; action facts allow a lemma to quantify over the relative order of individual compromise and session events, which the KCI lemmas require.

**Table 5.** Outcomes and cost aggregated by prover.

|                                  | ProVerif | Tamarin |
|----------------------------------|----------|---------|
| Properties posed                 | 12       | 6       |
| Proved                           | 6        | 5       |
| Attack found                     | 2        | 1       |
| Inconclusive                     | 1        | 0       |
| Non-terminating                  | 3        | 0       |
| Time to a verdict (s)            | <1–24    | 2–3     |
| Time spent without a verdict (s) | 900      | 0       |

## 6.2 Resource Profile

The two provers behave very differently. ProVerif decided 9 of its 12 properties within 24s and failed on the remaining three by exhausting the budget rather than by converging slowly. Tamarin decided all six of its lemmas, including both KCI levels, within 2–3s and 7–18 constraint-solving steps.

The three properties that ProVerif left undecided are agreement for the initiator (P5a) and both KCI levels (P6a, P6b). The failure on the KCI queries has an identifiable cause. Revealing the static private key of the responder makes every responder-side Diffie–Hellman value derivable, and the resulting clause set does not converge. Restricting the initiator to a single session reduced memory pressure but did not remove the divergence. Tamarin is less sensitive to this effect because its constraint solving proceeds backwards from the property and does not need to enumerate the large set of terms that the adversary can derive.

## 6.3 Guidance

ProVerif is the more suitable choice when the property is secrecy or a correspondence between honest parties, when many variants of one model must be verified, or when a global staging of adversary capabilities, such as a transition to quantum computation, is the natural formulation. In this setting it is fast and fully automatic. Tamarin is the more suitable choice when a long-term secret is compromised during the scenario rather than after it, when a property depends on the relative order of specific events, or when explicit mutable state is required. The KCI analysis of Sect. 4.3 meets all three conditions. These observations derive from a single protocol and do not constitute a general claim about the tools.

## 7 Limitations

**Symbolic abstraction.** Both models assume a Dolev–Yao adversary and perfect primitives, so they provide no guarantees about side channels, implementation defects or concrete security bounds. The Diffie–Hellman abstraction treats

every 32-byte value as  $g^t$  for some term  $t$  and thus excludes small-subgroup and invalid-curve behaviour, as well as the check for an all-zero shared value.

**Transcript hash.** The flat transcript hash binds the same data as the iterated formulation, but it assumes that the hash chain is collision-free as a whole instead of deriving this property from a single compression function. A collision attack on BLAKE2s would be invisible to this model, as it would be to any symbolic model.

**Undecided queries.** A non-terminating query is not a proof. It shows only that the prover could not settle the question within the budget, and it says nothing about the protocol. Tamarin decided two of the three properties that ProVerif left undecided, but agreement for the initiator (P5a) remains open. The ProVerif KCI queries were also attempted with the initiator restricted to one and to two sessions; these runs would have established the property only for that number of sessions, and they did not terminate either.

**Scope.** This work verifies the handshake but does not measure it: no latency or throughput figures are reported, and the analysis of Sect. 5 relies on no measurement. The data plane, the cookie mechanism and endpoint roaming lie outside the models.

**Relation to prior work.** Where our conclusions differ from those of prior analyses [6,9], the difference reflects modelling assumptions rather than new findings. Section 3 states these assumptions so that such differences can be traced.

## 8 Conclusion

We analysed the WireGuard handshake with ProVerif and Tamarin on a single transcription and obtained three findings. First, the contribution of the pre-shared key can be established by a controlled experiment: two models that differ in one parameter yield a proof and an attack against an adversary with a discrete-logarithm oracle. Second, KCI resistance has a precise location between the commitment to transport keys and the acceptance of traffic, and the static-ephemeral value  $se$  keeps these two events apart; establishing this required Tamarin, after the saturation of ProVerif had diverged. Third, the cost of post-quantum migration is not a single number: an in-band ML-KEM-768 initiation exceeds the IPv6 minimum-MTU payload by 68 bytes, and the cost appears in message size, path reachability and assumption strength at the same time. Reporting the undecided queries alongside the proofs also shows where the two provers diverge, which is the information needed to choose between them. Future work could extend the comparison to the cookie mechanism and to KEM-based handshakes, to determine whether the observed division of labour between the provers carries over to other protocols.

## References

1. Blanchet, B.: Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security* **1**(1–2), 1–135 (2016)
2. Cremers, C., Dax, A., Medinger, N.: Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols. In: *ACM SIGSAC Conference on Computer and Communications Security (CCS 2024)*, pp. 1046–1060. ACM (2024)
3. Deering, S., Hinden, R.: Internet Protocol, Version 6 (IPv6) Specification. RFC 8200 (2017)
4. Dolev, D., Yao, A.C.: On the security of public key protocols. *IEEE Transactions on Information Theory* **29**(2), 198–208 (1983)
5. Donenfeld, J.A.: WireGuard: Next generation kernel network tunnel. In: *Network and Distributed System Security Symposium (NDSS 2017)*. Internet Society (2017)
6. Donenfeld, J.A., Milner, K.: Formal verification of the WireGuard protocol. Tech. rep., University of Oxford (2018)
7. Fairhurst, G., Jones, T., Tüxen, M., Rüngeler, I., Völker, T.: Packetization Layer Path MTU Discovery for Datagram Transports. RFC 8899 (2020)
8. Hülsing, A., Ning, K.C., Schwabe, P., Weber, F., Zimmermann, P.R.: Post-quantum WireGuard. In: *IEEE Symposium on Security and Privacy (S&P 2021)*, pp. 304–321. IEEE (2021)
9. Lipp, B., Blanchet, B., Bhargavan, K.: A mechanised cryptographic proof of the WireGuard virtual private network protocol. In: *IEEE European Symposium on Security and Privacy (EuroS&P 2019)*. IEEE (2019)
10. Lowe, G.: A hierarchy of authentication specifications. In: *IEEE Computer Security Foundations Workshop (CSFW 1997)*. IEEE (1997)
11. Meier, S., Schmidt, B., Cremers, C., Basin, D.: The TAMARIN prover for the symbolic analysis of security protocols. In: *Computer Aided Verification (CAV 2013)*. LNCS, vol. 8044, pp. 696–701. Springer (2013)
12. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. FIPS 203 (2024)
13. Perrin, T.: The Noise protocol framework, revision 34 (2018), <https://noiseprotocol.org/noise.html>
14. Schmiege, S.: Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK. *Cryptology ePrint Archive*, Paper 2024/523 (2024), <https://eprint.iacr.org/2024/523>
15. Varner, K., Lipp, B., Zaeske, W., Schmidt, L.: Rosenpass: A formally verified post-quantum secure key exchange for WireGuard (2023), <https://rosenpass.eu>